

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/327866621>

# A statistical comparison for evaluating the effectiveness of linear and nonlinear manifold detection techniques for software defect prediction

Article in *International Journal of Advanced Intelligence Paradigms* · January 2018

DOI: 10.1504/IJAIP.2018.10016341

CITATIONS

15

3 authors, including:



Soumi Ghosh

26 PUBLICATIONS 614 CITATIONS

SEE PROFILE

READS

157



Ajay Rana

Amity University

308 PUBLICATIONS 2,431 CITATIONS

SEE PROFILE

---

## **A statistical comparison for evaluating the effectiveness of linear and nonlinear manifold detection techniques for software defect prediction**

---

Soumi Ghosh\* and Ajay Rana

Department of Computer Science and Engineering,  
Amity University Uttar Pradesh,  
Noida, Uttar Pradesh, India  
Email: soumighoshphd@gmail.com  
Email: ajay\_rana@amity.edu  
\*Corresponding author

Vineet Kansal

Department of Computer Science and Engineering,  
I.T.S. Engineering College,  
Greater Noida, Uttar Pradesh, India  
Email: dir.engg@its.edu.in

**Abstract:** Most of the software systems are released without predicting defects and therefore, this paper presents a new effective technique – manifold detection technique (MDT) is essential and different than earlier applied defect prediction methods like regression, feature selection methods, etc. In this paper, performance of classifiers has been compared with or without MDTs to evaluate the effectiveness of different MDTs (linear and nonlinear) by reducing the dimensions of software datasets. In this process, eight classifiers were applied to four PROMISE datasets to determine the best performing classifier with respect to prediction performance measuring factors (accuracy, precision, recall, F-measure, AUC, misclassification error) with or without MDTs. The experimental results statistically tested by paired two-tailed t-test proved that FastMVU is the most accurate result producing technique as compared to all other nonlinear MDTs and Bayesian network (BN) is the most effective technique for software defect prediction using with or without MDTs.

**Keywords:** defects; linear; nonlinear; manifold detection; promise datasets; prediction; software system.

**Reference** to this paper should be made as follows: Ghosh, S., Rana, A. and Kansal, V. (2019) 'A statistical comparison for evaluating the effectiveness of linear and nonlinear manifold detection techniques for software defect prediction', *Int. J. Advanced Intelligence Paradigms*, Vol. 12, Nos. 3/4, pp.370–391.

**Biographical notes:** Soumi Ghosh is pursuing her PhD in Computer Science and Engineering from the Amity University Uttar Pradesh, India. Her research areas include software engineering, data mining, machine learning, pattern recognition and fuzzy logic. She has published six research papers in some reputed national and international journals.

Ajay Rana is a Professor and the Director at the Amity University Uttar Pradesh, India. He received his PhD in Computer Science and Engineering from the U.P. Technical University, India. He is an Advisor and Founder Director of more than 39 different institutions and innovative programs at Amity Group. He is a member of board of governance (BoG), executive council (EC), advisory council (AC), board of studies (BoS) and special member of many Indian and foreign universities as well as industries. His research area includes software engineering. He has published 200 research papers in some reputed national and international journals. He is a member of CSI and IETE. He is the Editor in-Chief, technical committee member, advisory board member for 18 plus technical journals and conferences at national and international levels.

Vineet Kansal received his BTech in Computer Science and Engineering from the GB Pant University, Pantnagar in 1991. He received his MTech and PhD from the IIT Delhi in 1997 and 2003 respectively. He has over 25 years of teaching and academic administrative experience and he is currently working as a Professor and the Director at the I.T.S Engineering College, Greater Noida. He has over 21 international and national publications to his credit. He is a member of ACM, CSI and ISTE. He has been a Guest Faculty for many institutions including South Asian University and IMT Ghaziabad. He is a member, academic council and BoS CSE and IT at the Dr. APJ Abdul Kalam Technical University, Lucknow, UP, India.

This paper is a revised and expanded version of a paper entitled 'Software defect prediction system based on linear and nonlinear manifold detection' presented at INDIACOM-2017, BVICAM, New Delhi, India, 2 March 2017.

---

## **1 Introduction**

In software system, defect is a commonly observed phenomenon. It arises basically due to programming logic and (or) code which are not correct. Often we find that poor quality of software products and undesired level of output because of the defects. Defects in the software system results increase in development and maintenance expenses, failure in achieving the goals and ultimate dissatisfaction of the customers. For this reason, it is very crucial to find out means and ways for predicting defects in software system. If the defects can be detected in the early stage, then better results, output may be achieved and cost may also be reduced considerably. But mostly, it becomes rather difficult to identify such defects in the initial stage and even in the development stage too. As such, timely application of proper technique and accurate detection of defects has become necessary. Application of unsupervised nonlinear feature extraction techniques have been proposed and compared with linear techniques for better feature extraction (Backer et al., 1998). In recent years, many researchers have used various methods for prediction of software defects, but none of them are full proof or without limitations (Hall and Bowes, 2012). Feature selection technique along with the combination of independent variables have been used to make the fault prediction models more effective and thus, produce more desirable results (Hall et al., 2012). A combination of genetic algorithm and support vector machines (SVM) has been considered as an effective technique for inter-release fault prediction (Sarro et al., 2012). The two versions of NASA defect datasets have been

analysed and compared in order to find out the important differences and origin of the datasets used (Shepperd et al., 2013). Application of feature selection techniques showed effectiveness in terms of better bug prediction accuracy by virtue of use of different prediction models (Shivaji et al., 2013; Muthukumaran et al., 2015). A new technique for feature selection known as feature clustering and feature ranking (FECAR) along with FF-correlation measure has been proposed for prediction of software defects and investigate the rate of redundancy as well as studied the performance of different defect predictors (Liu et al., 2014). A technique called feature compression based on feature selection has been applied with the idea to detect defective modules in three successive versions of Eclipse datasets (Lu et al., 2014). An attempt has been made to prepare a framework for determining the fitness and performance of prediction of software defects. As well as comparison and evaluation of various prediction models that has been applied so far (Russo, 2014). Top k-metrics or minimum metric subset has been used with classification techniques in order to improve the accuracy of software defect prediction with limited resources (He et al., 2015). Classification and clustering techniques have been used for processing signals in biomedical applications (Dey and Ashour, 2016). With the basic need of a change in application of orthodox and standard methods in software analysis, it has been made mandatory to find out simple and meaningful settings of tuning for defect prediction with optimum outcome (Fu et al., 2016). A combination of supervised classification and unsupervised clustering technique has been proposed for the sake of diagnosis of disease related to coronary artery (Kausar et al., 2016). An automated system has been designed to perform hybridisation of K-nearest neighbour (K-NN), Naive Bayes (NB) and Apriori algorithm with rough datasets in order to assess of classifiers with machine learning (ML) (Ripon et al., 2016). Levenberg Marquardt back propagation network (LMBPN) classification technique has been proposed for detection and classification of fatty liver disease (Saba et al., 2016). For improvement of performance of classifiers in software defect prediction, automated parameter optimisation techniques have been observed to be considerably effective (Tantithamthavorn et al., 2016). SSA semi-supervised learning technique known as semi-supervised support vector machine (S3VM) with various kernel functions has been used for computer based diagnosis of disease (Zemmal et al., 2016). It has been observed through review of literature from 1992 to 2016 on this particular subject that SVM, ML and neural network (NN) techniques were widely applied by the researchers for the purpose of detection of defects in the software system (Ghosh et al., 2017). Nowadays, most of the software are released without detecting defects as software testers are having the idea that defect prediction is time consuming and pretty expensive. As such, there is a basic requirement of finding out technique, which is capable of predicting defects by means of eliminating erroneous, undesirable information. It has been observed that most of the datasets are having large dimensions and thereby containing defect prone datasets. As a result of this, it becomes mandatory to find out and use an effective technique which may very well reduce the dimensions of datasets and thus eliminate undesirable, error prone information. An attempt has been made in this work, to use manifold detection technique (MDT) in software defect prediction in such a way that it provides higher accuracy in less period of time. In this research, the prediction performance of all the classifiers with/without MDTs has been compared in order to determine the most effective technique for manifold detection and classifier with respect to higher accuracy, precision, recall, F-measure, AUC as well as lower misclassification error rate in case of software defect prediction. The basic objective of this research is to find out the merits

and usefulness of MDTs in this area. This paper has been arranged in four sections in such a way that, Section 2 narrates a detailed description of MDT (linear and nonlinear MDTs). In Section 3, methodology of experiment, its results are discussed and analysed. Statistical analysis of the performance of all the classifiers with/without MDTs has been performed by using paired two-tailed t-test. Finally, the paper has been concluded in Section 4.

## **2 Manifold detection technique**

This particular technique, MDT is associated with the task of transforming the datasets that are high-dimensional into a newer dataset having much lower dimensionality and in this process the geometry of the dataset also remain unchanged or unaltered. In other words, MDT is a technique which helps in transforming a dataset with high-dimensionality into a lower dimensionality without changing the meaningfulness as well as geometry of the basic dataset to a considerable extent. It has been found that dataset having reduced dimensionality basically relates to the intrinsic part of the dimensionality of the dataset. It also means that the datasets contain minimum parameters which are very much needed for the purpose of maintaining those properties of the dataset that are basic and well observed ones (Fukunaga, 1990). Manifold detection plays a very vital role in many domains, as it is very useful in classifying, visualising data and compressing high-dimensional data by virtue of a representation that is true and meaningful, reduction of processing time of computation and solving problem of high-dimensionality of datasets (Jimenez and Landgrebe, 1997). Further, it may be said that reduction in dimension may result separation of variables or features from those which are having less importance and hence more attention needs to be paid on the very pattern and characteristics of the dataset which may not be possible in any other way. MDT is really necessary, as it relates to projection and feature extraction also. Principal component analysis (PCA) (Pearson, 1901) and multi-dimensional scaling (MDS) (Cox and Cox, 2000) are important MDTs, which are often considered as most effective linear data transformation technique where as unsupervised nonlinear techniques such as isometric feature mapping (ISOMAP) (Tenenbaum et al., 2000), locally linear embedding (LLE) (Roweis and Saul, 2000), diffusion maps (Lafon and Lee, 2006), fast maximum variance unfolding (FastMVU) (Van der Maaten, 2007), stochastic proximity embedding (SPE) (Van der Maaten, 2007) and neighbourhood preserving embedding (NPE) (Van der Maaten, 2007) are most effective for manifold detection. These techniques largely depend on size of datasets, choice of neighbourhood and other parameters.

### *2.1 Linear MDT*

Linear manifold detection technique (linear MDT) means the components of low-dimensional vector are given by linear functions of components of corresponding high-dimensional datasets. In other words, high-dimensional variables of original dataset are replaced by a lower set of variables. Techniques like PCA and MDS) are very popular and significant linear MDT.

PCA (Pearson, 1901) originally known as Karhunen-Loeve, which is actually a linear technique for transforming dataset. This technique is very effective and at the same time

easy or simple in computation. It facilitates reduction in dimensionality and useful in finding out new, more informative and uncorrelated features of data. This technique is being actually considered as a popular linear technique for dimension reduction and classification of data (Virmani et al., 2016). It is effective for handling only those data which are inherently linear in nature. But it is not effective to represent nonlinear data in an accurate manner. PCA has another limitation, as it is not useful in discriminating data and hence, it may not be able to focus interesting viewpoint for clustering of data.

MDS (Cox and Cox, 2000; Kruskal, 1964) is considered as a linear MDT which can map high-dimensional dataset into low-dimensional one and at the same time retains the pair-wise distances between the data-points as much as possible. The quality of mapping is expressed in the form of a stress function which measures the errors between the pair-wise distances in both high and low-dimensional datasets. MDS is a very simple and effective technique for finding out hidden patterns and prediction of defects. An important drawback of this technique relates to a problem in case of selection of proper mapping dimension and also in case of indication of short distances of the pattern concerned. Apart from this, other drawbacks are that it needs huge computation and memory space.

## 2.2 *Nonlinear MDT*

In software system, MDTs like PCA and MDS are very suitable when the dataset is linear but such techniques are really unsuitable when the original high-dimensional dataset contains absolute nonlinear relationship among the variables. In that case, nonlinear MDTs (nonlinear MDTs) are more appropriate.

ISOMAP (Tenenbaum et al., 2000) is a technique that is nonlinear in nature and applied mainly for manifold detection, which also uses MDS technique with Geodesic or curvilinear distances between data-points. It has been observed that if the dataset is absolutely nonlinear then ISOMAP technique which is termed as non-iterative, polynomial-time algorithm is very effective for nonlinear manifold detection. For a specific class of data, this technique is effective converge to the true structure of the dataset asymptotically. Further, it is useful to compute data for a globally optimum solution. But, the important drawbacks of ISOMAP are its topological instability and its failure in presence of non-convex manifold. ISOMAP has high complexities like MDS and due to formation of erroneous connections in neighbourhood graph; its performance is considerably impaired.

LLE (Roweis and Saul, 2000) is a nonlinear MDT, computes low-dimensional dataset but at the same time preserve embeddings of high-dimensional inputs. It is similar to ISOMAP, as it represents the datasets on the basis of a graph. However, LLE in comparison to ISOMAP is not at all that sensitive to short circuiting because the properties of dataset are retained in this method. It facilitates the process of firmly fixing of even those manifolds which are not convex. LLE technique performs by finding a set of weights which makes local nonlinear interpolations that approximate the data closely.

Diffusion Maps (Lafon and Lee, 2006; Nadler et al., 2006) is actually a structure for manifold detection which is uniform, graphical and simultaneously performs as data parameter. This technique originates from the field of dynamical system and it is based on the work of giving an idea of what is Markov random walk on the graphical representation of data. Diffusion maps perform very well in case of a well sampled and clean dataset. When the dataset is having multiple sub-manifolds or additional noise then

it is not that effective one (Van der Maaten, 2007). Although this technique is having some similarities with ISOMAP, but as compared to ISOMAP, it is not that sensitive to short-circuiting because instead of taken into consideration only the shortest paths, Diffusion maps attempt to integrate all the paths or many paths through the graph.

FastMVU (Van der Maaten, 2007) has similarity with ISOMAP as it also defines a neighbourhood graph and at the same time pair-wise distances are also retained in the resulting graph. It starts functioning with preparation of a neighbourhood graph where each and every data-point becomes related with those closest neighbours. It differs from ISOMAP in the sense that it tries to unfold the dataset by maximising the Euclidean distances between data-points in such a manner that the distance in the neighbourhood graph remains unchanged or local geometry of the dataset is not distorted. By means of applying semi-definite programming, the resulting optimisation problem can be solved. So, it is also called as semi-definite embedding (SDE).

SPE (Van der Maaten, 2007) is a nonlinear MDT based on neighbourhood graph. It performs as iterative algorithm and it minimises the raw stress function of MDS technique. In the particular case of application of effective rule for updating the estimation of low-dimensional data representation, SPE differs from MDS. Further, it can be readily applied for the purpose of retaining distances in a neighbourhood graph that has been defined on the graph itself. As a result, such behaviour of SPE is very much comparable to ISOMAP.

NPE (Van der Maaten, 2007) is a nonlinear technique for manifold detection, which minimises the cost. But at the same time, NPE is having some particular problem like representation of data that turns linear while transformation occurs from high-dimensional to low-dimensional dataset. Moreover, it is also considered as linear version of LLE. In-fact, NPE like LLE also shows a graph of the neighbourhood data-points and thereafter the weights of reconstruction are computed with the linear combination of the data-point.

### 3 Methodology of experiment

In this experiment, MDP repository datasets (NASA/WVU IV and V, 2016) have been used along with eight different prediction classifiers such as multinomial logistic regression (MLR), NB, Bayesian network (BN), K-NN, alternating decision tree (ADT), C4.5 decision tree (C4.5 DT), random forest (RF), random tree (RT) for predicting the defects of software system.

#### 3.1 Datasets and experimental details

Four public datasets namely CM1, PC1, MW1 and KC3 have been collected from NASA MDP repository (NASA/WVU IV and V, 2016) for software defect prediction. The experiments have been performed on all the datasets using different classifiers with train/test data percentage is selected as 20/80. These datasets have been used to compare the performance measures of different classifiers with/without (linear and nonlinear) MDT. Table 1 shows the details of software defect dataset. Each dataset contains two labels (true or false) where True means defective modules and False means not-defective modules. All implementations of experiment have been performed in Waikato

environment for knowledge analysis (WEKA) toolkit and MATLAB using standard toolboxes and codes on Intel (R) Celeron (R) CPU N2807 processor 1.58GHz, 2GB RAM and Microsoft Windows 8 with 64 bit operating system.

### 3.2 Prediction performance measuring factors

The factors used for measuring prediction performance of all the classifiers are accuracy, precision, recall, F-measure, AUC and misclassification error. These factors have been calculated by using a confusion matrix as shown in Table 2. The definition and formulas of these factors have been given in Table 3.

**Table 1** Software defect dataset details

| <i>Name</i> | <i>Language</i> | <i>Total modules</i> | <i>Dimensions</i> | <i>Defect (%)</i> |
|-------------|-----------------|----------------------|-------------------|-------------------|
| CM1         | C               | 498                  | 22                | 9.839%            |
| PC1         | C               | 1,109                | 22                | 6.943%            |
| MW1         | C               | 403                  | 40                | 7.692%            |
| KC3         | Java            | 458                  | 40                | 9.389%            |

**Table 2** Confusion matrix for performance evaluation of classifiers

| <i>Actual value</i> | <i>Predicted value</i> |                     |
|---------------------|------------------------|---------------------|
|                     | <i>Not-defective</i>   | <i>Defective</i>    |
| Not-defective       | True negative (TN)     | False positive (FP) |
| Defective           | False negative (FN)    | True positive (TP)  |

**Table 3** Definition and formulas of factors

| <i>Factors</i>          | <i>Definition and formulas</i>                                                                                                                                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accuracy                | It is considered as correct classification rate and measures the possibility that defect proneness of module is predicted correctly.<br>Accuracy = $(TP + TN)/(TP + FP + FN + TP)$                                            |
| Precision               | It is also called as correctness and measures the possibility of predicting correctly the exact defective modules among those modules which are classified as defect-prone. Precision = $TP/(TP + FP)$                        |
| Recall                  | It is also termed as rate of detection of defects or probability of detection or true positive rate (TPR) or sensitivity. It determines the sensitivity of problem, product quality and completeness. Recall = $TP/(TP + FN)$ |
| F-measure               | It is calculated as a combined measure of recall as well as precision.<br>F-measure = $(2 * (recall * precision)) / (recall + precision)$                                                                                     |
| AUC                     | AUC stands for area under receiver operating characteristics (ROC) curve. By plotting the probability of false alarm (PF) on x-axis and Probability of detection (PD) on y-axis.                                              |
| Misclassification error | It is considered as incorrect classification rate and measures the possibility that defect proneness of module is predicted incorrectly. Misclassification Error = $(FN + FP)/(TP + FP + FN + TP)$                            |

### 3.3 *Cross-fold validation test*

In our experiment, ten -fold cross-validation test has been performed for evaluating the performance of defect prediction classifiers. In cross-validation test, the original dataset has been segregated into two different parts like training and testing set. For performing analysis, the training set is used where as testing set is applied for validating the results of analysis. We used cross validation test (ten-fold), where only one set is used as training and remaining nine are testing sets. In case of repeating this process for ten times, each set gets a chance to be used as training set.

### 3.4 *Experiments and results*

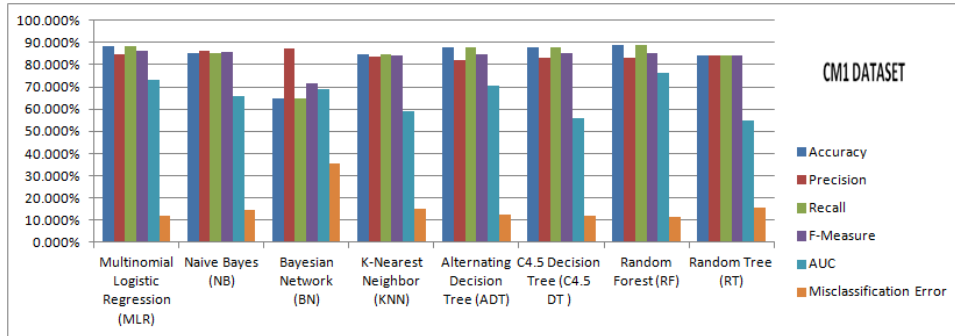
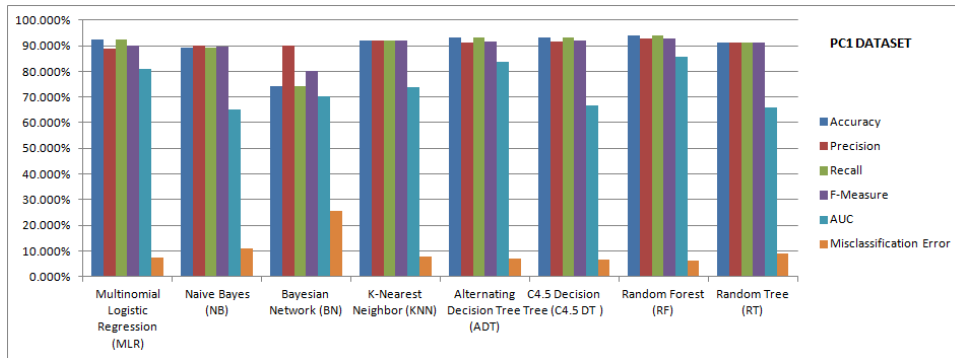
This experiment has been made in two separate parts, i.e.,

- 1 without MDT
- 2 with MDT.

In both the experiments, performance measures of various classifiers have been compared with/without MDTs in order to find out the most effective software defect prediction technique with higher accuracy rate and lower misclassification error rate.

#### 3.4.1 *Without MDT*

In this phase, all four datasets CM1, PC1, MW1 and KC3 have been applied to eight different classifiers. Application of different datasets presented different prediction performance measures of the said classifiers. Based on the prediction performance measuring values obtained for all the classifiers, a graphical representation has been made. The comparative performance measuring factors of all the classifiers without MDT on CM1, PC1, MW1 and KC3 dataset are shown in Figures 1, 2, 3, and 4. Tables 4, 5, 6, and 7 are given in Appendix shows the comparative analysis of all the classifiers only in terms of accuracy, misclassification error and recall on four datasets. For CM1 dataset, RF classifier showed higher level of accuracy (88.755%) and lower misclassification error rate (11.245%) where as MLR 88.153%, 11.847% and C4.5 DT 87.952% 12.048%. In PC1 dataset, accuracy and misclassification error rates are: RF 93.959%, 6.042%; C4.5 DT 93.327%, 6.673%; ADT 93.057%, 6.943%. Similarly, for MW1 dataset accuracy 91.5633% and misclassification error rate 8.4367% have been found in case of C4.5 DT classifier. But, when KC3 dataset is applied, ADT 90.393%, 9.607%; MLR 89.738%, 10.262%; RF 89.5197%, 10.4803%. However, based on the results of analysis, it was found that RF classifier performed well for CM1 and PC1 dataset but in case of MW1 and KC3 dataset, C4.5 DT and ADT outperforms for prediction of software defects in terms of level of accuracy and misclassification error.

**Figure 1** Comparative performance measuring factors of all classifiers without MDT on CM1 dataset (see online version for colours)**Figure 2** Comparative performance measuring factors of all classifiers without MDT on PC1 dataset (see online version for colours)

### 3.4.2 With MDT

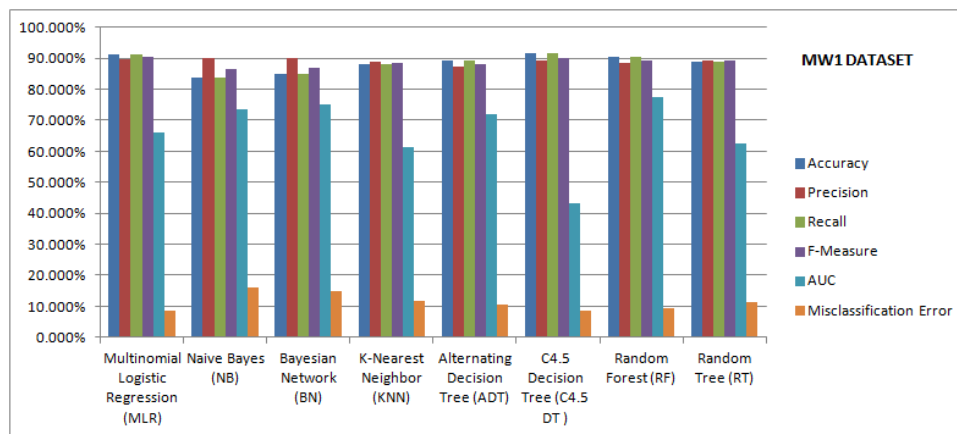
To evaluate the performance of the said classifiers, a new technique known as manifold detection has been applied on the same datasets for software defect prediction. The application of the technique has been segregated into two parts, i.e., linear MDTs and nonlinear MDTs.

#### 3.4.2.1 Linear MDTs

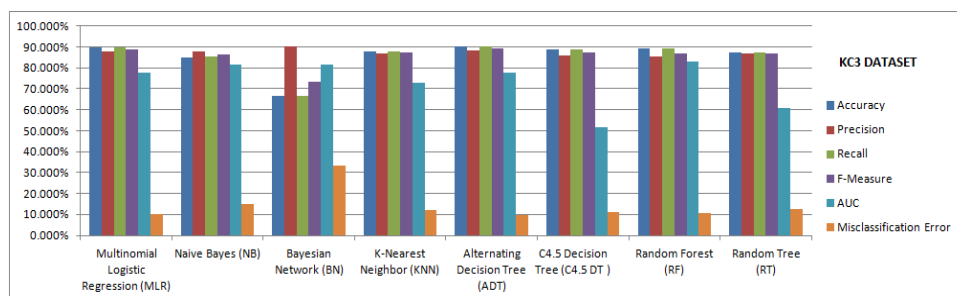
In this part, CM1, PC1, MW1, KC3 datasets having 22, 22, 40 and 40 dimensions have been reduced to only three dimensions (including class labels) using both PCA and MDS linear MDTs since an 'elbow curve' occurs at a point where embedding dimension equals to two (excluding the class labels). All the lower dimensional datasets have been classified and the results of eight classifiers obtained are used for prediction performance evaluation in terms of accuracy and misclassification error rate. Based on the prediction performance measuring values obtained for all the classifiers, a graphical representation has been made. After applying all linear MDTs on CM1, PC1, MW1 and KC3 dataset, the comparative performance measuring factors of all the classifiers are shown in Figures

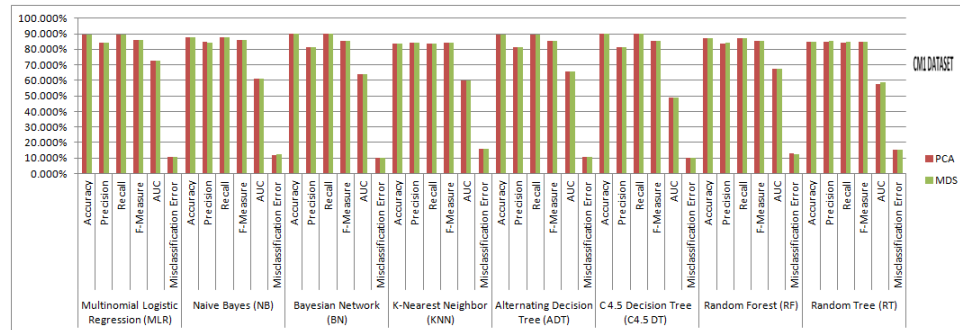
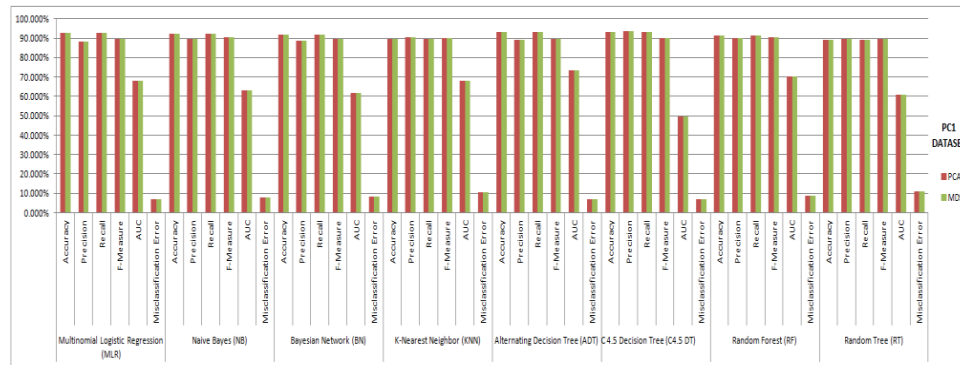
5, 6, 7, 8. Tables 4, 5, 6, 7 are given in shows the comparative analysis of all the classifiers only in terms of accuracy, misclassification error and recall on four datasets. For CM1 dataset, the higher accuracy and lower misclassification error rate of BN, C4.5 DT classifiers is 90.161% and 9.839% where as in PC1 dataset, 93.147% is higher accuracy level for C4.5 DT. The classifier is having lower misclassification error rate 6.853% respectively. Similarly, for MW1 dataset, C4.5 DT is the only classifier having 92.060% higher level of accuracy and lower misclassification error rate 7.940%. But, in KC3 dataset only BN classifier showed higher accuracy 90.175% and lower misclassification error rate 9.825% as compared to all other classifiers. Hence, analysing the results of prediction performance measuring factors of four datasets, it has been observed that C4.5 DT and BN classifiers provide higher accuracy and lower misclassification error rate for all lower dimensional datasets obtained after application of (PCA and MDS) linear MDTs and it outperforms all other classifiers in prediction of software defects.

**Figure 3** Comparative performance measuring factors of all classifiers without MDT on MW1 dataset (see online version for colours)

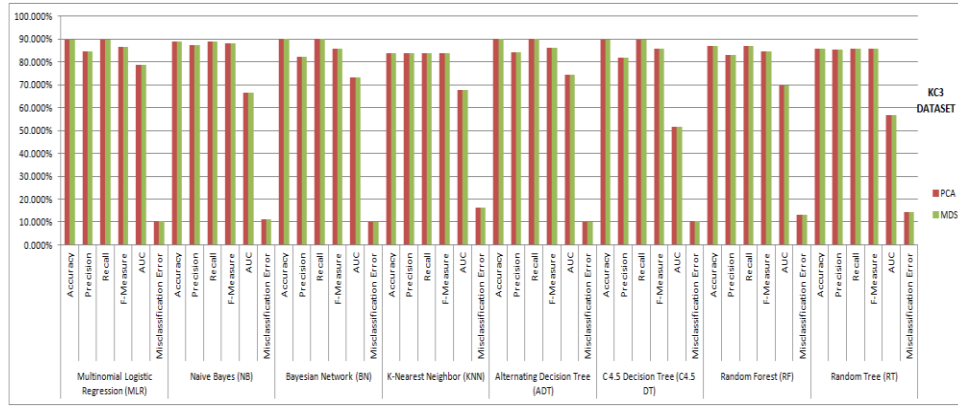


**Figure 4** Comparative performance measuring factors of all classifiers without MDT on KC3 dataset (see online version for colours)

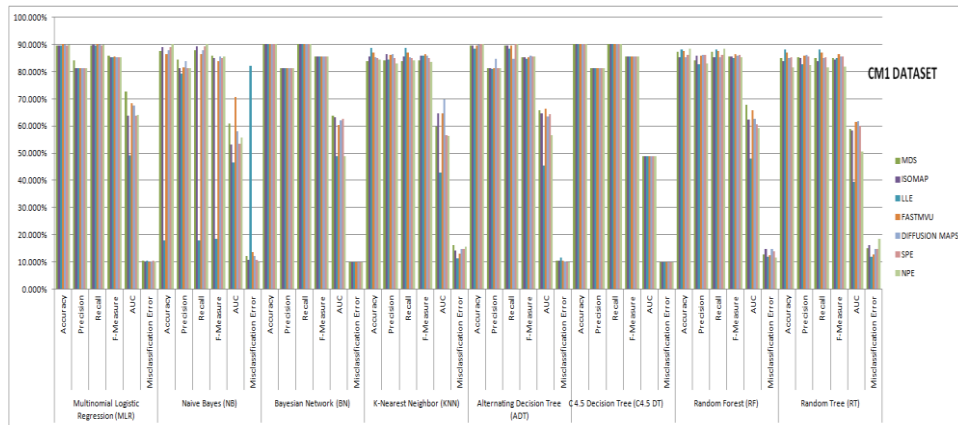


**Figure 5** Comparative performance measuring factors of all classifiers with linear MDTs on CM1 dataset (see online version for colours)**Figure 6** Comparative performance measuring factors of all classifiers with linear MDTs on PC1 dataset (see online version for colours)**Figure 7** Comparative performance measuring factors of all classifiers with linear MDTs on MW1 dataset (see online version for colours)

**Figure 8** Comparative performance measuring factors of all classifiers with linear MDTs on KC3 dataset (see online version for colours)

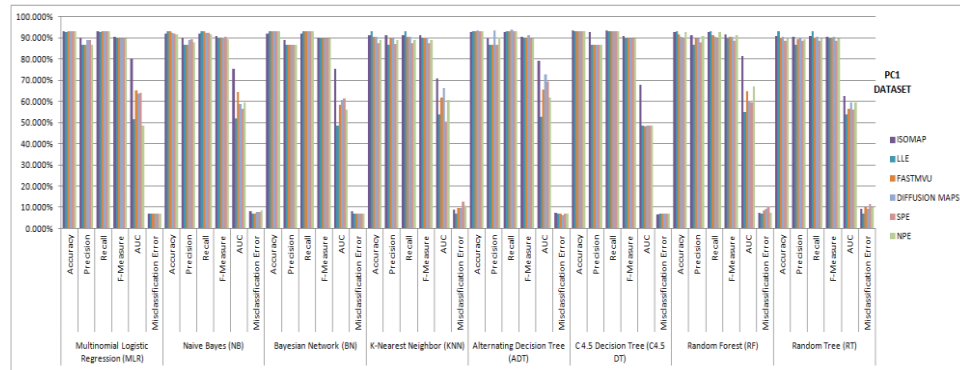
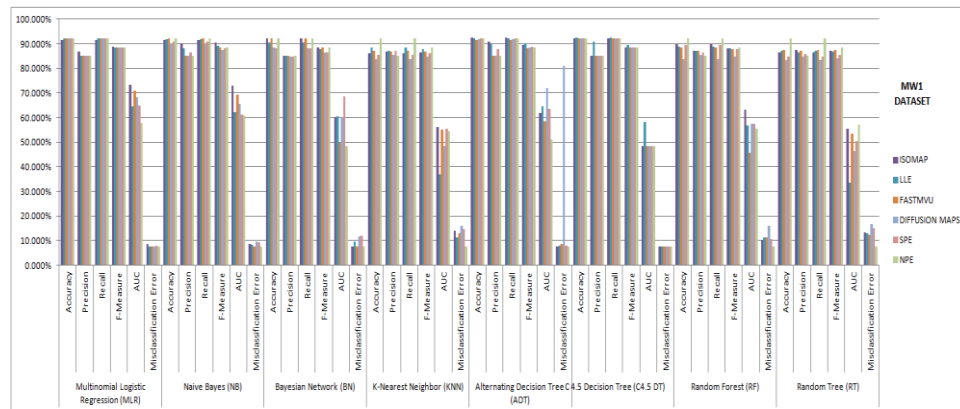
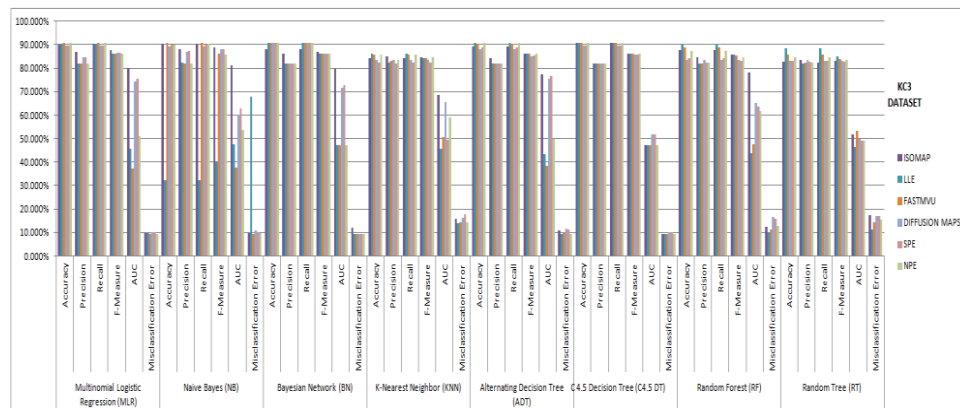


**Figure 9** Comparative performance measuring factors of all classifiers with nonlinear MDT on CM1 dataset (see online version for colours)



### 3.4.2.2 Nonlinear MDTs

Different nonlinear MDTs such as ISOMAP, LLE, FastMVU, DIFFUSION MAPS, SPE and NPE have been applied initially on the same high-dimensional datasets in order to extract only three-dimensional datasets since an ‘elbow curve’ occurs at a point where embedding dimension equals to two (excluding the class labels). Eight classification algorithms have been used separately to the lower-dimensional datasets and compared performance of all lower-dimensional datasets with different nonlinear MDTs in terms of accuracy, misclassification error, precision, recall, F-measure and AUC. Based on the prediction performance measuring values obtained for all the classifiers, a graphical representation has been made. After applying all nonlinear MDTs on CM1, PC1 MW1 and KC3 dataset, the comparative performance measuring factors of all the classifiers are shown in Figures 9, 10, 11, 12. Tables 4, 5, 6, 7 are given in Appendix shows the comparative analysis of all the classifiers only in terms of accuracy, misclassification error and recall on four datasets.

**Figure 10** Comparative performance measuring factors of all classifiers with nonlinear MDTs on PC1 dataset (see online version for colours)**Figure 11** Comparative performance measuring factors of all classifiers with nonlinear MDTs on MW1 dataset (see online version for colours)**Figure 12** Comparative performance measuring factors of all classifiers with nonlinear MDTs on KC3 dataset (see online version for colours)

### *3.5 Results analysis*

The results of experiments showed that the use of MDT in software defect prediction enables better visualisation of high dimensional datasets and minimum number of dimensions is also estimated for proper representation of software datasets by reducing dimensions, thereby eliminating unwanted, least meaningful information. MDTs also help in achieving higher level of accuracy in prediction performance of classifiers with lesser computation time. Therefore, application of MDTs in software defect prediction is more effective than the case without using MDTs. Linear MDTs are having considerable defects as they produce exactly similar results for all classifiers in terms of level of accuracy, misclassification error and other factors. That is why, linear MDTs can hardly be assumed as appropriate techniques for manifold detection. A comparative analysis of the outcome of application of nonlinear MDTs clearly showed that as compared to all other techniques, FastMVU is having more effectiveness and accuracy for manifold detection as a nonlinear and it is also having lower rate of misclassification error when applied in any defective software datasets. Comparing the performance of all the eight classifiers, it has also been demonstrated that C4.5 DT is result-oriented when applied on all the four datasets along with different nonlinear MDTs. BN algorithm also produces good result for all datasets with nonlinear MDTs but in case of LLE, Diffusion maps and SPE on MW1 dataset failed to show any desired result. As a result of this, C4.5 DT and BN when applied with any of the nonlinear MDTs have been considered as most effective classifiers for software defect prediction on the basis of output obtained.

### *3.6 Statistical analysis and testing of experimental results*

A paired two-tailed t-test has statistically tested the results of experiments in order to verify that whether prediction performance of all the classifiers using MDTs is significantly different than the case without using MDTs and to statistically analyse that BN outperforms other classifiers with/without MDTs. In-fact, the test performed by two-tailed t-test is based on comparison of all classifiers (pair-wise) with/without MDTs, which are used for prediction of software defects and performance of all the classifiers for each dataset are analysed at a specific level of significance ( $\alpha$ ) equals to 0.05. For each comparison, p value is computed and the results showed that p value is less than  $\alpha$  that causes statistically significant difference. Two-tailed t-test results are shown in Table 8 is given in Appendix. It has been statistically verified that the prediction performance of all classifiers except ADT and RT with MDTs are the most effective and outperform compared to the case of without using MDTs in software defect prediction. Based on the analysis of P-values, it is proved that BN is statistically more significant and different from other classifiers with/without MDTs. Further, NB is statistically significant with/without MDTs except in the case of LLE. ADT and RT classifiers have performed bad and showed no significant, notable differences in all the cases with/without MDTs.

## **4 Conclusions**

Problem of defects in the software system is very common and therefore, the task of detection of defect-prone software is rather crucial. Simultaneously, reduction of

high-dimensionality of datasets into lower one without any alteration of meaningfulness of original datasets has become an equally important task. This paper presents usage of MDT in predicting defects in software system by reducing high-dimensional datasets into low-dimensional datasets in such a way that reduces only data-size with minimum loss of information but the basic properties of the datasets are retained intact. For the basic purpose of reducing dimension of software datasets, MDT has been used for the task of prediction of software defect using four PROMISE datasets and eight different classifiers. The performance of various MDTs like PCA, MDS, ISOMAP, LLE, FastMVU, DIFFUSION MAPS, SPE and NPE have been evaluated and compared to determine the most effective technique for manifold detection and classifier among all the eight classifiers, which may produce the desired results with respect to higher accuracy, precision, recall, F-Measure, AUC as well as lowest misclassification error, when applied with/without MDTs. Based on the results of comparative analysis of prediction performance of all the classifiers applied with/without MDTs, it has been concluded that all the classifiers applied with different MDTs outperforms compared to the case of without using MDTs. Hence, in this experiment, the merits and usefulness of MDTs in case of software defect prediction is distinctly observed in the sense that application of MDT helps in better visualisation of high-dimensional data and minimum number of dimensions are also estimated to represent the software datasets by reducing the number of dimensions along with lesser computation time for classification of datasets. The results showed that linear MDTs cannot be considered as appropriate technique for manifold detection but at the same time, nonlinear MDTs also cannot perform well and produce good results unless the manifolds contain well sampled data-points and un-scattered datasets. Therefore, nonlinear MDTs proved to be better techniques for manifold detection than linear MDTs in case of software defect prediction. The experimental results proved that FastMVU is the most accurate result producing technique as compared to all other nonlinear MDTs, when applied with any defective software dataset. The experiment also demonstrated that BN classifier is the most effective and desired performing prediction technique in regard to higher accuracy level and lower misclassification error rate, irrespective of any software defective dataset in case of with/without MDTs. The performance of all the classifiers with/without MDTs has been statistically analysed and tested by performing paired two-tailed t-test. This technique may very well be used in other real world applications too in addition to the task of software defect prediction.

## References

- Backer, S.D., Naud, A. and Scheunders, P. (1998) 'Non-linear dimensionality reduction techniques for unsupervised feature extraction', *Pattern Recognition Letters*, Vol. 19, No. 8, pp.711–720.
- Cox, T.F. and Cox, M.A. (2000) *Multi-Dimensional Scaling, in Monographs on Statistics and Applied Probability*, 2nd ed., pp.1–328, Chapman and Hall/CRC, New York, ISBN: 978-1584880943.
- Dey, N. and Ashour, A. (2016) *Classification and Clustering in Biomedical Signal Processing*, pp.1–463, IGI Publishing Hershey, PA, USA, ISBN: 1522501401 9781522501404.
- Fu, W., Menzies, T. and Shen, X. (2016) 'Tuning for software analytics: is it really necessary?', *Information and Software Technology*, Vol. 76, pp.135–146.
- Fukunaga, K. (1990) *Introduction to Statistical Pattern Recognition*, 2nd ed., pp.1–592, Academic Press Professional, Inc., San Diego, CA, USA, ISBN: 0-12-26-985 1-7.

- Ghosh, S., Rana, A. and Kansal, V. (2017) 'Predicting defect of software system', *Proceedings of the 5th International Conference on Frontiers in Intelligent Computing: Theory and Applications FICTA*, Vol. 516, pp.55–67.
- Hall, T. and Bowes, D. (2012) 'The state of machine learning methodology in software fault prediction', *Proceedings of the 11th International Conference on Machine Learning and Applications ICMLA*.
- Hall, T., Beecham, S., Bowes, D., Gray, D. and Counsell, S. (2012) 'A systematic literature review on fault prediction performance in software engineering', *IEEE Transactions on Software Engineering*, Vol. 38, No. 6, pp.1276–1304.
- He, P., Li, B., Liu, X., Chen, J. and Ma, Y. (2015) 'An empirical study on software defect prediction with a simplified metric set', *Information and Software Technology*, Vol. 59, pp.170–190.
- Jimenez, L.O. and Landgrebe, D.A. (1997) 'Supervised classification in high-dimensional space: geometrical, statistical and asymptotical properties of multivariate data', *IEEE Transactions on Systems, Man and Cybernetics – TSMC*, Vol. 28, No. 1, pp.39–54.
- Kausar, N., Abdullah, A., Samir, B.B., Palaniappan, S., AlGhamdi, B.S. and Dey, N. (2016) 'Ensemble clustering algorithm with supervised classification of clinical data for early diagnosis of coronary artery disease', *Journal of Medical Imaging and Health Informatics*, American Scientific Publishers, Vol. 6, No. 1, pp.78–87.
- Kruskal, J.B. (1964) 'Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis', *Psychometrika*, Vol. 29, No. 1, pp.1–27.
- Lafon, S. and Lee, A.B. (2006) 'Diffusion maps and coarse-graining: a unified framework for dimensionality reduction, graph partitioning and data set parameterization', *IEEE Transaction on Pattern Analysis and Machine Intelligence*, IEEE Computer Society, Washington, DC, USA, Vol. 28, No. 9, pp.1393–1403.
- Liu, S., Chen, X., Liu, W., Chen, J., Gu, Q. and Chen, D. (2014) 'FECAR: a feature selection framework for software defect prediction', *IEEE 38th Annual Computer Software and Applications Conference COMPSAC*, pp.426–435.
- Lu, H., Kocaguneli, E. and Cukic, B. (2014) 'Defect prediction between software versions with active learning and dimensionality reduction', *IEEE 25th International Symposium on Software Reliability Engineering ISSRE*, pp.312–322.
- Muthukumaran, K., Rallapalli, A. and Bhanu Murthy, N.L. (2015) 'Impact of feature selection techniques on bug prediction models', *Proceedings of the 8th India Software Engineering Conference ISEC*, pp.120–129.
- Nadler, B., Lafon, S., Coifman, R.R. and Kevrekidis, I.G. (2006) 'Diffusion maps, spectral clustering and the reaction coordinates of dynamical systems', *Applied and Computational Harmonic Analysis: Special Issue on Diffusion Maps and Wavelets*, Vol. 21, pp.113–127.
- NASA/WVU IV and V Facility, Metrics Data Program (MDP) (2016) [online] <http://mdp.ivv.nasa.gov> (accessed 10 February 2016).
- Pearson, K. (1901) 'On lines and planes of closest fit to systems of points in space', *Philosophical Magazine*, Vol. 2, pp.559–572 [online] <http://pbil.univ-lyon1.fr/R/pearson1901.pdf> (accessed 20 January 2016).
- Ripon, S.H., Kamal, S., Hossain S. and Dey N. (2016) 'Theoretical analysis of different classifiers under reduction rough data set: a brief proposal', *International Journal of Rough Sets and Data Analysis (IJRSDA)*, Vol. 3, No. 3, pp.1–20.
- Roweis, S.T. and Saul, L.K. (2000) 'Nonlinear dimensionality reduction by locally linear embedding', *Science*, Vol. 290, No. 5500, pp.2323–2326.
- Russo, B. (2014) 'A proposed method to evaluate and compare fault predictions across studies', *PROMISE Proceedings of the 10th International Conference on Predictive Models in Software Engineering*, pp.2–11.

- Saba, L., Dey, N., Ashour, A.S., Samanta, S., Nath, S.S., Chakraborty, S., Sanches, J., Kumar, D., Marinho, R.T. and Suri, J.S. (2016) 'Automated stratification of liver disease in ultrasound: an online accurate feature classification paradigm', *Computer Methods and Programs in Biomedicine*, Elsevier, Vol. 130, pp.118–134.
- Sarro, F., Martino, S. Di, Ferrucci, F. and Gravino, C. (2012) 'A further analysis on the use of genetic algorithm to configure support vector machines for inter-release fault prediction', *Proceedings of the 27th Annual ACM Symposium on Applied Computing SAC*, pp.1215–1220.
- Shepperd, M.J., Song, Q., Sun, Z. and Mair, C. (2013) 'Data quality: some comments on the nasa software defect datasets', *IEEE Transactions on Software Engineering*, Vol. 39, No. 9, pp.1208–1215.
- Shivaji, S., Whitehead, E.J., Akella Jr, R. and Kim, S. (2013) 'Reducing features to improve code change-based bug prediction', *IEEE Transactions on Software Engineering*, Vol. 39, No. 4, pp.552–569.
- Tantithamthavorn, C., McIntosh, S., Hassan, A.E. and Matsumoto, K. (2016) 'Automated parameter optimization of classification techniques for defect prediction models', *Proceedings of the 38th International Conference on Software Engineering ICSE*, pp.321–332.
- Tenenbaum, J.B., Silva, V.D. and Langford, J. (2000) 'A global geometric framework for nonlinear dimensionality reduction', *Science*, Vol. 29, No. 5500, pp. 2319–2323.
- Van der Maaten, L.J.P. (2007) *An Introduction to Dimensionality Reduction Using Matlab*, Report MICC, Maastricht University, Netherlands, pp.1–44 [online] [http://tsam-fich.wdfiles.com/localfiles/apuntes/Report\\_final.pdf](http://tsam-fich.wdfiles.com/localfiles/apuntes/Report_final.pdf) (accessed 4 March 2016).
- Virmani, J.K., Dey, N. and Kumar, V. (2016) 'PCA-PNN and PCA-SVM based CAD systems for breast density classification', *Applications of Intelligent Optimization in Biology and Medicine: Current Trends and Open Problems*, Vol. 96, pp.159–180, Springer International Publishing, Springer, Cham.
- Zemmal, N., Azizi, N., Dey, N. and Sellami, M. (2016) 'Adaptive semi supervised support vector machine semi supervised learning with features cooperation for breast cancer classification', *Journal of Medical Imaging and Health Informatics*, American Scientific Publishers, Vol. 6, No. 1, pp.53–62.

## Appendix

Table 4 Comparative prediction performance analysis of all classifiers in CM1 dataset

| Technique                   | Parameter                   | Without MDTs | Linear MDTs | ISOMAP | LLE    | FASTMVU | DIFFUSION MAPS | SPE    | NPE    |
|-----------------------------|-----------------------------|--------------|-------------|--------|--------|---------|----------------|--------|--------|
| MLR                         | Accuracy (%)                | 88.153       | 89.558      | 89.759 | 89.558 | 90.161  | 89.960         | 89.558 | 89.960 |
|                             | Recall                      | 0.882        | 0.896       | 0.898  | 0.896  | 0.902   | 0.9            | 0.896  | 0.9    |
| NB                          | Misclassification error (%) | 11.847       | 10.442      | 10.241 | 10.442 | 9.839   | 10.040         | 10.442 | 10.040 |
|                             | Accuracy (%)                | 85.341       | 87.952      | 89.157 | 17.872 | 86.546  | 87.952         | 89.157 | 90.161 |
| BN                          | Recall                      | 0.853        | 0.88        | 0.892  | 0.179  | 0.865   | 0.88           | 0.892  | 0.902  |
|                             | Misclassification error (%) | 14.659       | 12.048      | 10.843 | 82.129 | 13.454  | 12.048         | 10.843 | 9.839  |
| KNN                         | Accuracy (%)                | 64.659       | 90.161      | 90.161 | 90.161 | 90.161  | 90.161         | 90.161 | 90.161 |
|                             | Recall                      | 0.647        | 0.902       | 0.902  | 0.902  | 0.902   | 0.902          | 0.902  | 0.902  |
| ADT                         | Misclassification error (%) | 35.341       | 9.839       | 9.839  | 9.839  | 9.839   | 9.839          | 9.839  | 9.839  |
|                             | Accuracy (%)                | 84.739       | 83.936      | 85.743 | 88.755 | 86.948  | 85.341         | 85.141 | 84.337 |
| C4.5 DT                     | Recall                      | 0.847        | 0.839       | 0.857  | 0.888  | 0.869   | 0.853          | 0.851  | 0.843  |
|                             | Misclassification error (%) | 15.261       | 16.064      | 14.257 | 11.245 | 13.052  | 14.659         | 14.859 | 15.663 |
| RF                          | Accuracy (%)                | 87.751       | 89.558      | 89.558 | 88.554 | 89.558  | 89.960         | 90.161 | 90.161 |
|                             | Recall                      | 0.878        | 0.896       | 0.896  | 0.886  | 0.896   | 0.847          | 0.902  | 0.902  |
| RT                          | Misclassification error (%) | 12.249       | 10.442      | 10.442 | 11.446 | 10.442  | 10.040         | 9.839  | 9.839  |
|                             | Accuracy (%)                | 87.952       | 90.161      | 90.161 | 90.161 | 90.161  | 90.161         | 90.161 | 90.161 |
| C4.5 DT                     | Recall                      | 0.88         | 0.902       | 0.902  | 0.902  | 0.902   | 0.902          | 0.902  | 0.902  |
|                             | Misclassification error (%) | 12.048       | 9.839       | 9.839  | 9.839  | 9.839   | 9.839          | 9.839  | 9.839  |
| RF                          | Accuracy (%)                | 88.755       | 87.149      | 85.341 | 88.153 | 87.550  | 85.341         | 86.145 | 88.554 |
|                             | Recall                      | 0.888        | 0.871       | 0.853  | 0.882  | 0.876   | 0.853          | 0.861  | 0.886  |
| RT                          | Misclassification error (%) | 11.245       | 12.851      | 14.659 | 11.847 | 12.450  | 14.659         | 13.855 | 11.446 |
|                             | Accuracy (%)                | 84.337       | 84.538      | 83.936 | 88.153 | 87.149  | 85.141         | 85.341 | 81.526 |
| Misclassification error (%) | Recall                      | 0.843        | 0.845       | 0.839  | 0.882  | 0.871   | 0.851          | 0.853  | 0.815  |
|                             | Misclassification error (%) | 15.663       | 15.462      | 16.064 | 11.847 | 12.851  | 14.859         | 14.659 | 18.474 |

Note: Italic indicates highest accuracy and lowest misclassification error rate of classifiers

**Table 5** Comparative prediction performance analysis of all classifiers in PC1 dataset

| Technique | Parameter                   | Without MDTs | Linear MDTs | ISOMAP | LLE    | FASTMVU | DIFFUSION MAPS | SPE    | NPE    |
|-----------|-----------------------------|--------------|-------------|--------|--------|---------|----------------|--------|--------|
| MLR       | Accuracy (%)                | 92.426       | 92.847      | 92.967 | 92.877 | 93.057  | 92.967         | 92.967 | 93.057 |
|           | Recall                      | 0.924        | 0.929       | 0.93   | 0.929  | 0.931   | 0.93           | 0.93   | 0.931  |
| NB        | Misclassification error (%) | 7.574        | 7.124       | 7.033  | 7.124  | 6.943   | 7.033          | 7.033  | 6.943  |
|           | Accuracy (%)                | 89.179       | 92.245      | 91.885 | 93.057 | 93.057  | 92.245         | 92.155 | 91.524 |
| BN        | Recall                      | 0.892        | 0.922       | 0.919  | 0.931  | 0.931   | 0.922          | 0.922  | 0.915  |
|           | Misclassification error (%) | 10.821       | 7.755       | 8.115  | 6.943  | 6.943   | 7.755          | 7.845  | 8.476  |
| KNN       | Accuracy (%)                | 74.391       | 91.704      | 92.065 | 93.057 | 93.057  | 93.057         | 93.057 | 93.057 |
|           | Recall                      | 0.744        | 0.917       | 0.921  | 0.931  | 0.931   | 0.931          | 0.931  | 0.931  |
| ADT       | Misclassification error (%) | 25.609       | 8.296       | 7.935  | 6.943  | 6.943   | 6.943          | 6.943  | 6.943  |
|           | Accuracy (%)                | 92.065       | 89.630      | 91.073 | 92.967 | 90.352  | 90.532         | 87.556 | 89.089 |
| C4.5 DT   | Recall                      | 0.921        | 0.896       | 0.911  | 0.93   | 0.904   | 0.905          | 0.876  | 0.891  |
|           | Misclassification error (%) | 7.935        | 10.370      | 8.927  | 7.033  | 9.648   | 9.468          | 12.444 | 10.911 |
| RF        | Accuracy (%)                | 93.057       | 92.967      | 92.696 | 93.057 | 93.057  | 93.688         | 93.057 | 93.057 |
|           | Recall                      | 0.931        | 0.93        | 0.927  | 0.931  | 0.931   | 0.937          | 0.931  | 0.931  |
| RT        | Misclassification error (%) | 6.943        | 7.033       | 7.304  | 6.943  | 6.943   | 6.312          | 6.943  | 6.943  |
|           | Accuracy (%)                | 93.327       | 93.147      | 93.418 | 93.057 | 93.057  | 93.057         | 93.057 | 93.057 |
| C4.5 DT   | Recall                      | 0.933        | 0.931       | 0.934  | 0.931  | 0.931   | 0.931          | 0.931  | 0.931  |
|           | Misclassification error (%) | 6.673        | 6.853       | 6.583  | 6.943  | 6.943   | 6.943          | 6.943  | 6.943  |
| RF        | Accuracy (%)                | 93.959       | 91.434      | 92.786 | 92.967 | 91.434  | 90.622         | 89.811 | 92.786 |
|           | Recall                      | 0.94         | 0.914       | 0.928  | 0.93   | 0.914   | 0.906          | 0.898  | 0.928  |
| RT        | Misclassification error (%) | 6.042        | 8.566       | 7.214  | 7.033  | 8.566   | 9.378          | 10.189 | 7.214  |
|           | Accuracy (%)                | 91.073       | 88.999      | 90.712 | 92.967 | 90.171  | 90.622         | 88.458 | 89.901 |
| C4.5 DT   | Recall                      | 0.911        | 0.89        | 0.907  | 0.93   | 0.902   | 0.906          | 0.885  | 0.899  |
|           | Misclassification error (%) | 8.927        | 11.001      | 9.288  | 7.033  | 9.829   | 9.378          | 11.542 | 10.099 |

Note: Italic indicates highest accuracy and lowest misclassification error rate of classifiers

**Table 6** Comparative prediction performance analysis of all classifiers in MW1 dataset

| Technique | Parameter                   | Without MDTs | Linear MDTs | ISOMAP | LLE    | FASTM/U | DIFFUSION MAPS | SPE    | NPE    |
|-----------|-----------------------------|--------------|-------------|--------|--------|---------|----------------|--------|--------|
| MLR       | Accuracy (%)                | 91.315       | 91.315      | 91.563 | 92.308 | 92.308  | 92.308         | 92.060 | 92.308 |
|           | Recall                      | 0.913        | 0.913       | 0.916  | 0.923  | 0.923   | 0.923          | 0.921  | 0.923  |
|           | Misclassification error (%) | 8.685        | 8.685       | 8.437  | 7.692  | 7.692   | 7.692          | 7.940  | 7.692  |
| NB        | Accuracy (%)                | 83.871       | 90.074      | 91.563 | 91.811 | 92.308  | 90.323         | 90.819 | 92.308 |
|           | Recall                      | 0.839        | 0.901       | 0.916  | 0.918  | 0.923   | 0.903          | 0.908  | 0.923  |
|           | Misclassification error (%) | 16.129       | 9.926       | 8.437  | 8.189  | 7.692   | 9.677          | 9.181  | 7.692  |
| BN        | Accuracy (%)                | 85.112       | 91.315      | 92.308 | 90.571 | 92.308  | 88.338         | 88.089 | 92.308 |
|           | Recall                      | 0.851        | 0.913       | 0.923  | 0.906  | 0.923   | 0.883          | 0.881  | 0.923  |
|           | Misclassification error (%) | 14.888       | 8.685       | 7.692  | 9.429  | 7.692   | 11.663         | 11.91  | 7.692  |
| KNN       | Accuracy (%)                | 88.089       | 86.601      | 86.104 | 88.586 | 87.097  | 83.871         | 85.360 | 92.308 |
|           | Recall                      | 0.881        | 0.866       | 0.861  | 0.886  | 0.871   | 0.839          | 0.854  | 0.923  |
|           | Misclassification error (%) | 11.911       | 13.400      | 13.896 | 11.414 | 12.903  | 16.129         | 14.640 | 7.692  |
| ADT       | Accuracy (%)                | 89.330       | 90.323      | 92.556 | 92.060 | 91.563  | 91.811         | 92.060 | 92.308 |
|           | Recall                      | 0.893        | 0.903       | 0.926  | 0.921  | 0.916   | 0.918          | 0.921  | 0.923  |
|           | Misclassification error (%) | 10.670       | 9.677       | 7.444  | 7.940  | 8.437   | 81.189         | 7.940  | 7.692  |
| C4.5 DT   | Accuracy (%)                | 91.563       | 92.060      | 92.308 | 92.556 | 92.308  | 92.308         | 92.308 | 92.308 |
|           | Recall                      | 0.916        | 0.921       | 0.923  | 0.926  | 0.923   | 0.923          | 0.923  | 0.923  |
|           | Misclassification error (%) | 8.437        | 7.940       | 7.692  | 7.444  | 7.692   | 7.692          | 7.692  | 7.692  |
| RF        | Accuracy (%)                | 90.571       | 90.323      | 89.826 | 88.834 | 88.586  | 83.871         | 89.578 | 92.308 |
|           | Recall                      | 0.906        | 0.903       | 0.898  | 0.888  | 0.886   | 0.839          | 0.896  | 0.923  |
|           | Misclassification error (%) | 9.429        | 9.677       | 10.174 | 11.166 | 11.414  | 16.129         | 10.422 | 7.692  |
| RT        | Accuracy (%)                | 88.834       | 85.112      | 86.601 | 87.097 | 87.593  | 83.375         | 84.864 | 92.308 |
|           | Recall                      | 0.888        | 0.851       | 0.866  | 0.871  | 0.876   | 0.834          | 0.849  | 0.923  |
|           | Misclassification error (%) | 11.166       | 14.888      | 13.400 | 12.903 | 12.407  | 16.625         | 15.137 | 7.692  |

Note: Italic indicates highest accuracy and lowest misclassification error rate of classifiers

**Table 7** Comparative prediction performance analysis of all classifiers in KC3 dataset

| <i>Technique</i> | <i>Parameter</i>            | <i>Without MDTs</i> | <i>Linear MDTs</i> | <i>ISOMAP</i> | <i>LLE</i> | <i>FASTMVU</i> | <i>DIFFUSION MAPS</i> | <i>SPE</i> | <i>NPE</i> |
|------------------|-----------------------------|---------------------|--------------------|---------------|------------|----------------|-----------------------|------------|------------|
| MLR              | Accuracy (%)                | 89.738              | 89.738             | 90.175        | 90.393     | 90.611         | 89.738                | 89.738     | 90.611     |
|                  | Recall                      | 0.897               | 0.897              | 0.902         | 0.904      | 0.906          | 0.897                 | 0.897      | 0.906      |
| NB               | Misclassification error (%) | 10.262              | 10.262             | 9.825         | 9.607      | 9.389          | 10.262                | 10.262     | 9.389      |
|                  | Accuracy (%)                | 85.153              | 88.865             | 90.393        | 32.096     | 90.611         | 89.301                | 89.956     | 90.175     |
| BN               | Recall                      | 0.852               | 0.889              | 0.904         | 0.321      | 0.906          | 0.893                 | 0.9        | 0.902      |
|                  | Misclassification error (%) | 14.847              | 11.135             | 9.607         | 67.904     | 9.389          | 10.699                | 10.044     | 9.825      |
| KNN              | Accuracy (%)                | 66.594              | 90.175             | 87.991        | 90.611     | 90.611         | 90.611                | 90.611     | 90.611     |
|                  | Recall                      | 0.666               | 0.902              | 0.88          | 0.906      | 0.906          | 0.906                 | 0.906      | 0.906      |
| ADT              | Misclassification error (%) | 33.406              | 9.825              | 12.009        | 9.389      | 9.389          | 9.389                 | 9.389      | 9.389      |
|                  | Accuracy (%)                | 87.773              | 83.843             | 84.280        | 86.026     | 85.808         | 83.625                | 82.314     | 85.808     |
| C4.5 DT          | Recall                      | 0.878               | 0.838              | 0.843         | 0.86       | 0.858          | 0.836                 | 0.823      | 0.858      |
|                  | Misclassification error (%) | 12.227              | 16.157             | 15.721        | 13.974     | 14.192         | 16.376                | 17.686     | 14.192     |
| RF               | Accuracy (%)                | 90.393              | 89.956             | 89.301        | 90.611     | 90.393         | 88.210                | 88.865     | 90.611     |
|                  | Recall                      | 0.904               | 0.9                | 0.893         | 0.906      | 0.904          | 0.882                 | 0.889      | 0.906      |
| RT               | Misclassification error (%) | 9.607               | 10.044             | 10.699        | 9.389      | 9.607          | 11.790                | 11.135     | 9.389      |
|                  | Accuracy (%)                | 88.865              | 89.738             | 90.611        | 90.611     | 90.611         | 89.738                | 89.738     | 90.611     |
| RF               | Recall                      | 0.889               | 0.897              | 0.906         | 0.906      | 0.906          | 0.897                 | 0.897      | 0.906      |
|                  | Misclassification error (%) | 11.135              | 10.262             | 9.389         | 9.389      | 9.389          | 10.262                | 10.262     | 9.389      |
| RT               | Accuracy (%)                | 87.336              | 85.808             | 82.533        | 88.646     | 85.808         | 82.969                | 83.188     | 84.498     |
|                  | Recall                      | 0.873               | 0.858              | 0.825         | 0.886      | 0.858          | 0.83                  | 0.832      | 0.845      |
| RT               | Misclassification error (%) | 12.664              | 14.192             | 17.467        | 11.354     | 14.192         | 17.031                | 16.812     | 15.502     |
|                  | Accuracy (%)                | 87.336              | 85.808             | 82.533        | 88.646     | 85.808         | 82.969                | 83.188     | 84.498     |
| RT               | Recall                      | 0.873               | 0.858              | 0.825         | 0.886      | 0.858          | 0.83                  | 0.832      | 0.845      |
|                  | Misclassification error (%) | 12.664              | 14.192             | 17.467        | 11.354     | 14.192         | 17.031                | 16.812     | 15.502     |

Note: Italic indicates highest accuracy and lowest misclassification error rate of classifiers

**Table 8** Results of paired two-tailed t-test of without/with MDTs

| <i>Techniques</i>                     | <i>Without MDTs vs linear MDTs</i> | <i>Without MDTs vs nonlinear MDTs</i> |                    |                    |                       |                                       |
|---------------------------------------|------------------------------------|---------------------------------------|--------------------|--------------------|-----------------------|---------------------------------------|
|                                       |                                    | <i>ISOMAP</i>                         | <i>LLE</i>         | <i>FASTMVU</i>     | <i>DIFFUSION MAPS</i> | <i>SPE</i> <i>NPE</i>                 |
| Multinomial logistic regression (MLR) | 0.1311                             | 0.0516                                | 0.0123             | 0.0169             | 0.0584                | 0.0516   0.0121                       |
| Naive Bayes (NB)                      | Not significant                    | Not significant                       | <i>Significant</i> | <i>Significant</i> | Not significant       | Not significant <i>Significant</i>    |
|                                       | 0.0082                             | 0.0101                                | 0.1273             | 0.0258             | 0.0088                | 0.0061   0.0129                       |
| Bayesian network (BN)                 | <i>Significant</i>                 | <i>Significant</i>                    | Not significant    | <i>Significant</i> | <i>Significant</i>    | <i>Significant</i> <i>Significant</i> |
|                                       | 0.0125                             | 0.0098                                | 0.0136             | 0.0100             | 0.0196                | 0.0204   0.0100                       |
| K-nearest neighbour (KNN)             | <i>Significant</i>                 | <i>Significant</i>                    | <i>Significant</i> | <i>Significant</i> | <i>Significant</i>    | <i>Significant</i> <i>Significant</i> |
|                                       | 0.0247                             | 0.1215                                | 0.2479             | 0.2843             | 0.0692                | 0.0486   0.4355                       |
| Alternating decision tree (ADT)       | <i>Significant</i>                 | Not significant                       | Not significant    | Not significant    | Not significant       | <i>Significant</i> Not significant    |
|                                       | 0.1743                             | 0.2167                                | 0.1140             | 0.1223             | 0.2583                | 0.2195   0.0805                       |
| C 4.5 decision tree (C4.5 DT)         | Not significant                    | Not significant                       | Not significant    | Not significant    | Not significant       | Not significant   Not significant     |
|                                       | 0.0948                             | 0.0438                                | 0.0597             | 0.0691             | 0.0894                | 0.0894   0.0691                       |
| Random forest (RF)                    | Not significant                    | <i>Significant</i>                    | Not significant    | Not significant    | Not significant       | Not significant   Not significant     |
|                                       | 0.0250                             | 0.0264                                | 0.1040             | 0.0154             | 0.0058                | 0.0208   0.3118                       |
| Random tree (RT)                      | <i>Significant</i>                 | <i>Significant</i>                    | Not significant    | <i>Significant</i> | <i>Significant</i>    | <i>Significant</i> Not significant    |
|                                       | 0.0574                             | 0.0797                                | 0.1672             | 0.4231             | 0.1071                | 0.0673   0.3066                       |
|                                       | Not significant                    | Not significant                       | Not significant    | Not significant    | Not significant       | Not significant   Not significant     |

Note: Italic indicates classifiers are significant